Ejercicio 1

```
def lee_batallas(fichero):
    with open(fichero, encoding='utf-8') as f:
        lector = csv.reader(f, delimiter = ";")
        next(lector)
        res = []
        for nombre, rey_atacante, rey_atacado, gana_atacante, muertes_principales,
                comandantes_atacantes, comandantes_atacados, region,
                num_atacantes, num_atacados in lector:
            gana_atacante = gana_atacante == 'win'
            muertes_principales = muertes_principales == '1'
            comandantes_atacantes = parsea_comandantes(comandantes_atacantes)
            comandantes_atacados = parsea_comandantes(comandantes_atacados)
            num_atacantes, num_atacados = parsea_num(num_atacantes),
                parsea_num(num_atacados)
            res.append(BatallaGOT(nombre, rey_atacante, rey_atacado, gana_atacante,
                muertes_principales, comandantes_atacantes, comandantes_atacados,
                region, num_atacantes, num_atacados))
        return res

def parsea_comandantes(comandantes):
    if comandantes:
        comandantes = [c.strip() for c in comandantes.split(",")]
    else:
        comandantes = []
    return comandantes

def parsea_num(num):
    if num:
        return int(num)
    else:
        return None
```

Módulo test_got

```
def test_lee_batallas():
    batallas = lee_batallas('../data/battles.csv')
    print(f"Leídas {len(batallas)} batallas.")
    print("Mostrando las 3 primeras:")
    for b in batallas[:3]:
        print("\t", b)
    print("Mostrando las 3 últimas:")
    for b in batallas[-3:]:
        print("\t", b)

if __name__=="__main__":
    batallas = test_lee_batallas()
```

Ejercicio 2

```
def reyes_mayor_menor_ejercito(batalla):
    participantes = {}
    for b in batalla:
        if b.num_atacantes != None:
            if b.rey_atacante not in participantes:
                participantes[b.rey_atacante] = 0
            participantes[b.rey_atacante] += b.num_atacantes
        if b.num_atacados != None:
            if b.rey_atacado not in participantes:
                participantes[b.rey_atacado] = 0
            participantes[b.rey_atacado] += b.num_atacados

    return max(participantes.items(), key = lambda t:t[1])[0],\
        min(participantes.items(), key = lambda t:t[1])[0]
```

Módulo test_got

```
def test_reyes_mayor_menor_ejercito(batallas):
    mayor, menor = reyes_mayor_menor_ejercito(batallas)
    print(f"El rey con el ejército mayor es {mayor}, y con el ejército menor es
{menor}.")

if __name__=="__main__":
    test_reyes_mayor_menor_ejercito(batallas)
```

Ejercicio 3

```
def batallas_mas_comandantes(batallas, regiones=None, n=None):
    res = sorted(((b.nombre,
        len(b.comandantes_atacantes) + len(b.comandantes_atacados))
        for b in batallas if regiones == None or b.region in regiones),
        key = lambda t:t[1], reverse = True)
    if n != None:
        return res[:n]
    else:
        return res
```

Módulo test_got

```
def test_batallas_mas_comandantes(batallas):
    print("Las 5 batallas con más comandantes:")
    print("\t", batallas_mas_comandantes(batallas,n=5))
    print("Todas las batallas en 'The North' o en 'The Riverlands' ordenadas de más a
menos comandantes participantes:")
    print("\t", batallas_mas_comandantes(batallas, {'The North', 'The Riverlands'}))

if __name__=="__main__":
    test_batallas_mas_comandantes(batallas)
```

Ejercicio 4

```
def rey_mas_victorias(batallas, rol='ambos'):
    conteos = {}
    for b in batallas:
        if b.rey_atacante not in conteos:
            conteos[b.rey_atacante] = 0
        if b.rey_atacado not in conteos:
            conteos[b.rey_atacado] = 0

        if (rol == 'ambos' or rol == 'atacante') and b.gana_atacante:
            conteos[b.rey_atacante] += 1
        elif (rol == 'ambos' or rol == 'atacado') and not b.gana_atacante:
            conteos[b.rey_atacado] += 1

    rey, conteo = max(conteos.items(), key = lambda t:t[1])
    if conteo > 0:
        return rey
    else:
        return None
```

Solución alternativa:

```
def rey_mas_victorias(batallas, rol='ambos'):
    if rol == 'ambos':
        contador1 = Counter(b.rey_atacante for b in batallas if b.gana_atacante)
        contador2 = Counter(b.rey_atacado for b in batallas
           if not b.gana_atacante)
        res = (contador1 + contador2)
    elif rol == 'atacante':
        res = Counter(b.rey_atacante for b in batallas if b.gana_atacante)
    else:
        res = Counter(b.rey_atacado for b in batallas if not b.gana_atacante)

    if len(res) > 0:
        return res.most_common(1)[0][0]
    else:
        return None
```

Módulo test_got

```
def test_rey_mas_victorias(batallas):
    print("Rey que ha ganado más batallas:", rey_mas_victorias(batallas))
    print("Rey que ha ganado más batallas como atacante:", rey_mas_victorias(batallas,
rol = "atacante"))
    print("Rey que ha ganado más batallas como atacado:", rey_mas_victorias(batallas,
rol = "atacado"))

if __name__=="__main__":
    test_rey_mas_victorias(batallas)
```

Ejercicio 5

```
def rey_mas_victorias_por_region(batallas, rol='ambos'):
    res = {}
    for b in batallas:
        if b.region in res:
            res[b.region].append(b)
        else:
            res[b.region] = [b]

    for region, lista in res.items():
        res[region] = rey_mas_victorias(lista, rol)

    return res
```

Solución alternativa:

```
def rey_mas_victorias_por_region(batallas, rol='ambos'):
    regiones = {b.region for b in batallas}
    res = {}
    for region in regiones:
        res[region] = rey_mas_victorias((b for b in batallas
            if b.region == region), rol)

    return res
```

Módulo test_got

```
def test_rey_mas_victorias_por_region(batallas):
    print("Reyes que han ganado más batallas en cada región:")
    for region, rey in rey_mas_victorias_por_region(batallas).items():
        print("\t",region,"=>",rey)
    print("Reyes que han ganado más batallas como atacantes en cada región:")
    for region, rey in rey_mas_victorias_por_region(batallas, rol = "atacante").items():
        print("\t",region,"=>",rey)
    print("Reyes que han ganado más batallas como atacados en cada región:")
    for region, rey in rey_mas_victorias_por_region(batallas, rol = "atacado").items():
        print("\t",region,"=>",rey)

if __name__=="__main__":
    test_rey_mas_victorias_por_region(batallas)
```